MILESTONE 1

Multi-Robot Simulation

Failure-case detection in a grid-world pickup simulation

1 OBJECTIVE

Demonstrate failure-case detection in the simulation *without* introducing communication delays or full robot behaviors. Specifically, show that the simulation correctly identifies every illegal condition.

2 IMPLEMENTATION

GUI library: `PySide6`

Files

FILE	RESPONSIBILITY
`main.py`	Main GUI window: map layout, status tab, buttons and their logic, and the mode parser.
`map_widget.py`	Grid map object, iteration logic, and map interactions (adding robots and goals). Sends sensor data back to each robot and updates goal pickup counts when items are retrieved.
`robot.py`	Robot object: ID, current position, actions (turn left, turn right, move forward, pick up), sensing, and data transmission. Keeps reading and sending queues for future development (TBD).
`goal.py`	Goal object: ID, position, and total pickup count.
`scenarios.py`	Sample scenarios for testing both failure and success cases.

3 GRAPHICAL INTERFACE

The window is split in two: the grid map on the left, and the control buttons and status panel on the right.

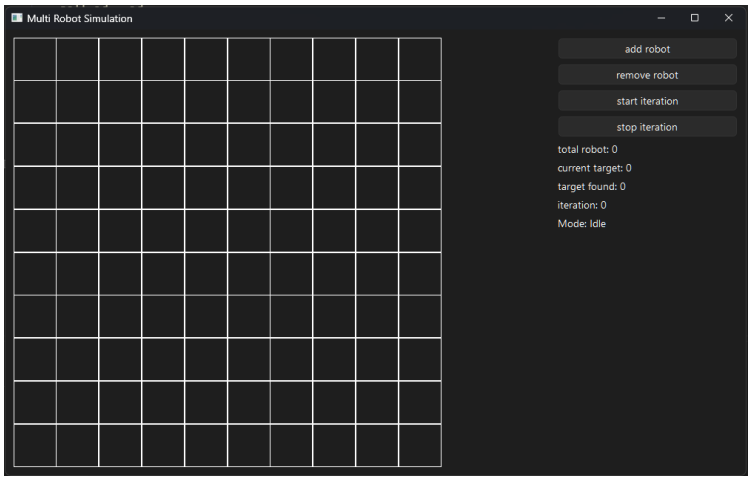


Figure 1. Idle state. Map on the left; controls and status on the right.

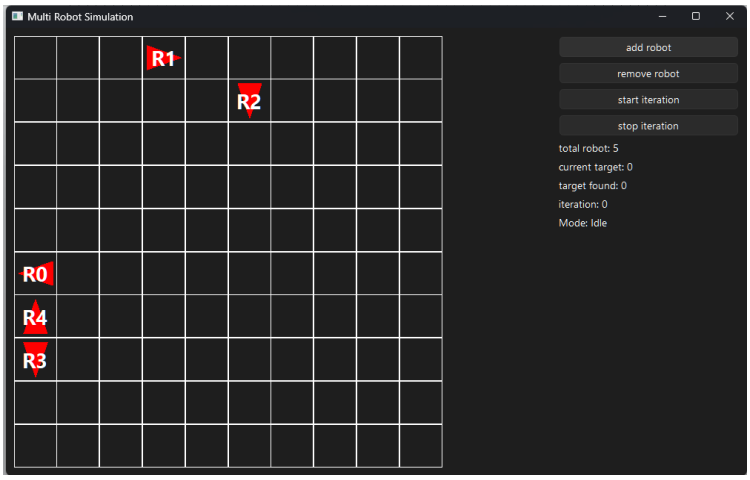


Figure 2. After adding robots, a red triangle spawns at a random cell, labeled with its ID.

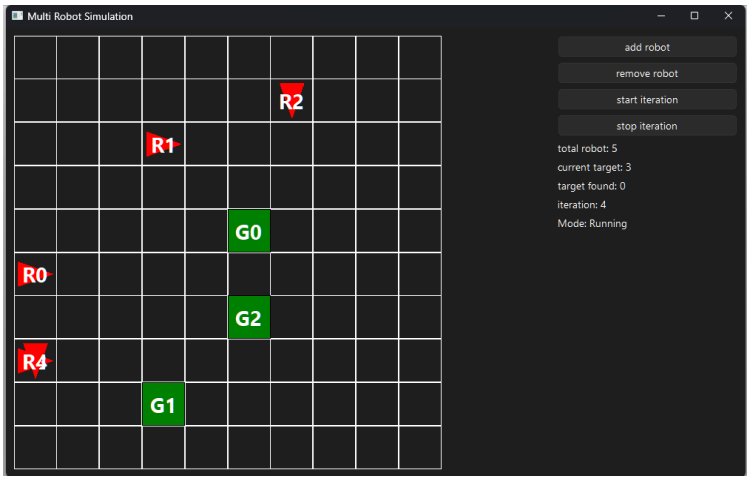


Figure 3. After starting, three goals appear at random cells. Once all three are collected, three new goals are generated.

4 COMMAND-LINE PARSER

ARGUMENT	BEHAVIOR
`--mode random`	Each robot picks its action randomly.
`--mode userinput`	Each iteration, a command (left / right / forward / pickup) is requested from the user for every robot. Robots and goals still spawn randomly.
`--scenario <name>`	Runs one pre-scripted scene with fixed goal positions, robot positions, and actions. Available: two_pick, empty_pick, one_pick, three_pick.

5 ILLEGAL-BEHAVIOR REPORTS

The simulation raises one of the following reports whenever a pickup violates the two-robot rule:

```
"Illegal behavior: more than two robots picked goal {goal.id}"

"Illegal behavior: only one robot picked goal {goal.id}"

"Illegal behavior: robot {robot_id} picked a goal at {position} but there is no
goal there"
```

6 ITERATION LOOP

Every map iteration executes the following step:

```
STEP
  Increment iteration

  IF no goals AND not scenario mode
      Generate 3 random goals

  FOR each robot
      Read messages
      Sense goals / robots around it

  FOR each robot
      Choose action based on mode
      Execute action
      Keep robot inside grid
      IF pick_up
          Record pickup
          Update goal

  FOR each goal
      IF picked by exactly 2 robots
          Remove goal
          Count as found
      ELSE IF illegal pickup
          Reset / report illegal behavior

  FOR each robot
      Send messages

  Update display
END STEP
```

7 FAILURE CASES

Each named scenario replays a fixed setup: fixed goal and robot cells, a scripted action list per robot instead of a decision, no new goals, and an automatic start. Every setup drives the map into exactly one of the conditions above, so the expected console output is known before the run.

SCENARIO	SETUP	CONSOLE OUTPUT AT THE DECIDING STEP
two_pick *control (legal)*	G0 at (5, 5). R0 (5, 6) facing N, R1 (4, 5) facing E. Both: forward, pick_up.	`Goal 0 picked successfully by two robots`
one_pick *under-pick*	G0 at (2, 2). R0 starts on the goal cell, facing N. Script: pick_up.	`Illegal behavior: only one robot picked goal 0`
three_pick *over-pick*	G0 at (5, 5). R0 (5, 6) N, R1 (4, 5) E, R2 (6, 5) W. All: forward, pick_up.	`Illegal behavior: more than two robots picked goal 0`
empty_pick *phantom pick*	G0 at (0, 0). R0 (3, 3) facing N on an empty cell. Script: pick_up.	`Illegal behavior: robot 0 picked a goal at [3, 3] but there is no goal there`

Pickup counts are evaluated at the end of every step: exactly two picks removes the goal and counts it as found, while any other non-zero count is reported and the counter is reset to 0, leaving the goal on the map for the next step. A pickup on a cell holding no goal is reported separately, after the goal checks.

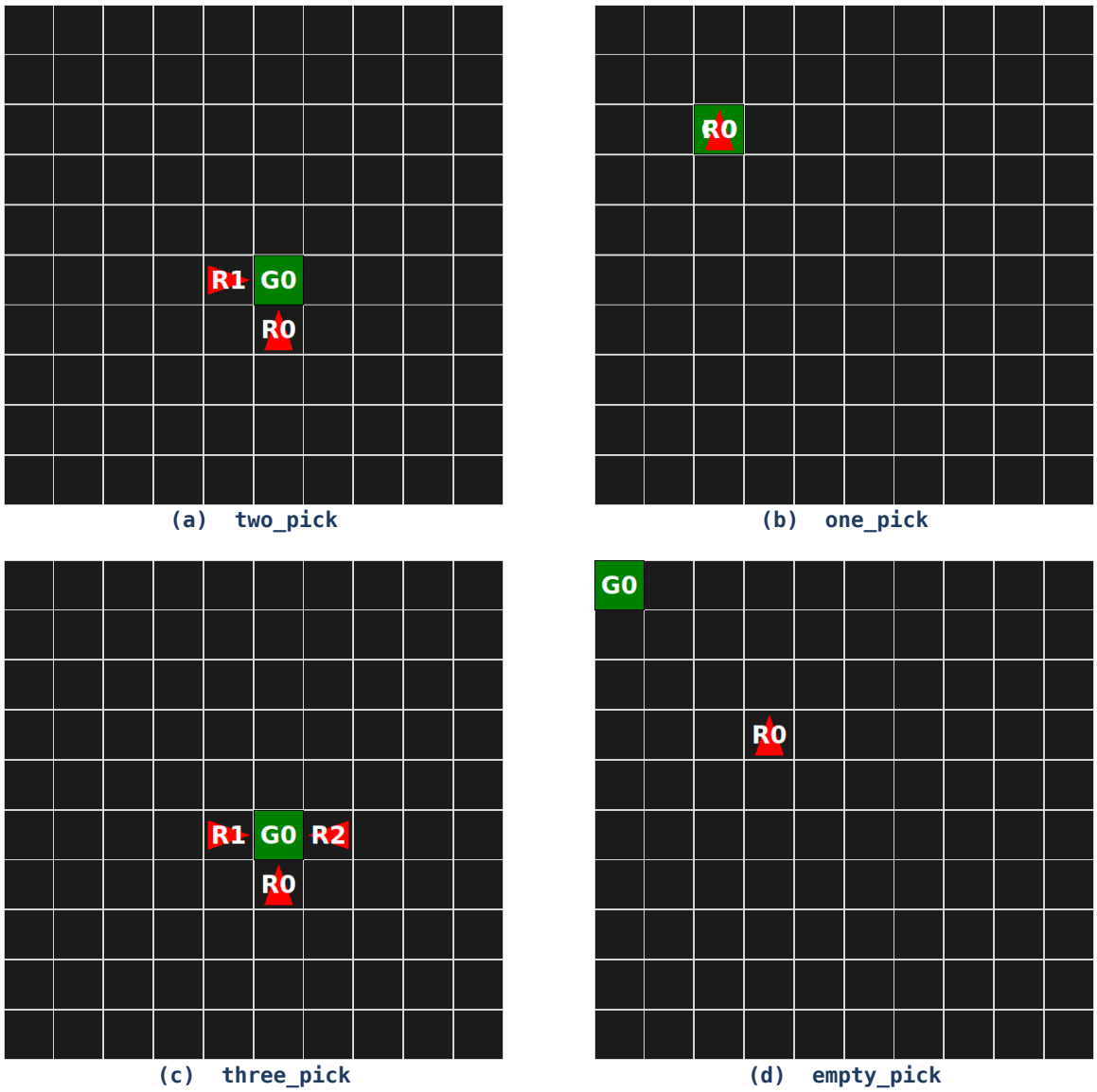


Figure 4. Start state of each scenario, before the first step. Green cells are goals, red triangles are robots pointing in their facing direction.